# Robot stack for OpenArm 2

Experiments, insights and proposal

26 October 2026

## Summary

### Insights from the experiments

1. The motion expert executes its first command well, but rarely follows a change of instruction and keeps no memory of earlier steps.
2. A trained noise policy makes the robot follow a change of instruction twice as often, with no target point and no change to the motion expert. Stopping, setting down, retracting and re-commanding is the most reliable redirect inside full missions.
3. A world model helps at the level of subtasks, not motions: imagining the outcome of each subtask lets the robot reach goals shown as a photo and complete far more missions.
4. Memory works outside the motion expert: an activity log for interruptions, rules for standing instructions, counters for what the camera cannot see.
5. Qwen3-VL-4B is the stronger of the two planners tested; picture goals and standing instructions need the world model and rules memory.

### Proposal

1. **Motion expert**: LingBot-VLA 2.0, compared with $\pi_{0.5}$ and GR00T in simulation and on the OpenArm 2 data.
2. **Steering**: the noise policy as the learned steering module, with stop–set down–retract as the fallback.
3. **World model**: beside the planner, imagining the outcome of each candidate subtask.
4. **Memory**: activity log, rules, counters, video memory and a progress monitor.
5. **Planner**: Qwen3-VL-4B as the baseline; fine-tuned and larger planners to evaluate.
6. **Data and training**: one flywheel where the robot's logs train every module.

# Part I Experiments and insights

## 1 The motion expert

### G0.5 in our tests

- **Fixed commands**: on LIBERO, a set of simulated tabletop tasks, it succeeds 97–99% of the time when the instruction stays the same.
- **Change of instruction**: when the instruction changes 2 s into a task, it completes the new task only 13–29% of the time (§2).
- **No memory**: it acts on the current camera frame only; when the instruction is removed after 2 s, success falls to 40% (§4).

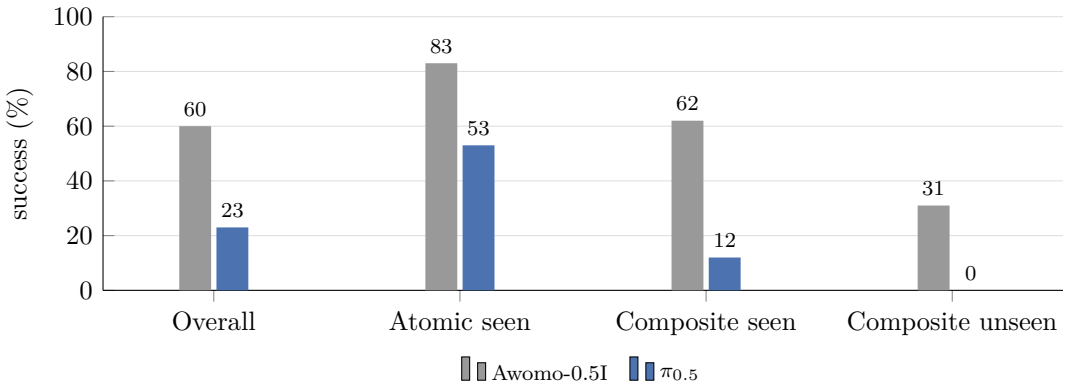



**Figure 1:** RoboCasa365 kitchen benchmark, 50 tasks × 2 trials. "Atomic": one skill; "composite": several skills chained; "unseen": not in training.

### Benchmark

- On the same 50 kitchen tasks, Awomo-0.5I succeeds 60% of the time and $\pi_{0.5}$ 23%: the choice of motion expert matters.
- Composite tasks are the hardest, unseen ones most of all (31% and 0%).

### Insights

- The motion expert is reliable on short, fixed commands.
- Changing tasks, remembering and chaining long tasks need modules around it: steering, memory and a planner that splits work into short subtasks.

### What the literature shows

G0.5 [3] reaches 98.9% on LIBERO. $\pi_{0.5}$ [1] and $\pi_{0.7}$ [2] pair a VLM with a flow-matching action expert and take subtask instructions from a high level. Awomo-0.5I [4] is a strong RoboCasa365 [28] policy.

## 2 Steering: changing the task mid-motion

### The problem

The robot is reaching for the bowl and the user says “no, the bottle”. A motion expert keeps doing its first task.

### The idea: a noise policy

The motion expert turns random noise into a motion. A small network that sees the cameras, the robot state and the new instruction chooses that noise, so the expert’s own motion heads toward the new task. The expert itself is never modified. The network is trained in two stages:

1. imitate the noise of the robot’s successful rollouts (DSBC);
2. improve it through a learned world model and a success critic, back-propagating through the expert’s sampler (SuperMimic-style).

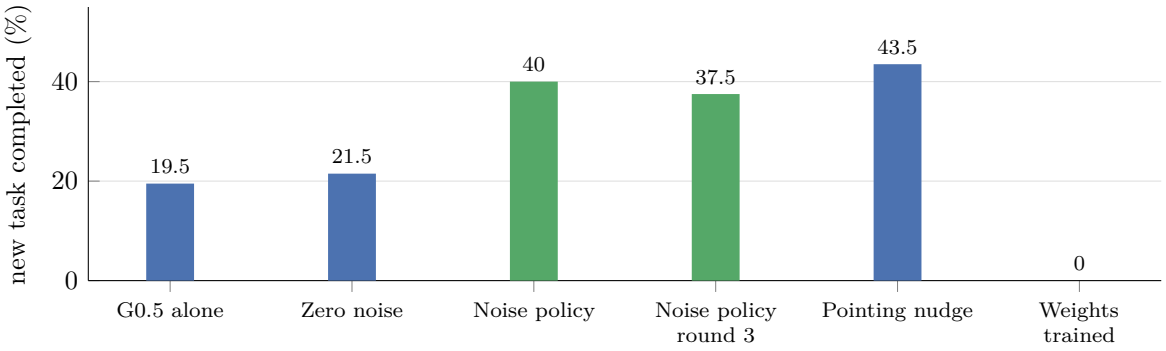



**Figure 2:** Instruction changed after 2 s; 200 trials per bar on start states not used for training.

### Results: instruction changed after 2 s

- The noise policy completes **40%** of new tasks, against 19.5% for G0.5 alone.
- This matches the pointing nudge (43.5%), a 2 s push toward the new target, without needing any target point.
- Zero noise gives 21.5%: the gain comes from what the policy sees.
- A third training round with fresh data stays at the same level (37.5%).
- Training the expert’s own weights with the same objective gives 0% and also breaks its normal tasks.

| Method | Target point | Training | Changes the expert | Result |
|---|---|---|---|---|
| Repeat the new command | no | no | no | 13–29% |
| Pointing nudge (person or VLM) | yes | no | no | 39–49% |
| **Noise policy** | **no** | per skill | **no** | **40%** |
| Train the expert’s weights | no | yes | yes | 0% |
| Stop, set down, retract, re-command | no | no | no | 96% in missions |

**Table 1:** Ways to redirect the motion expert.

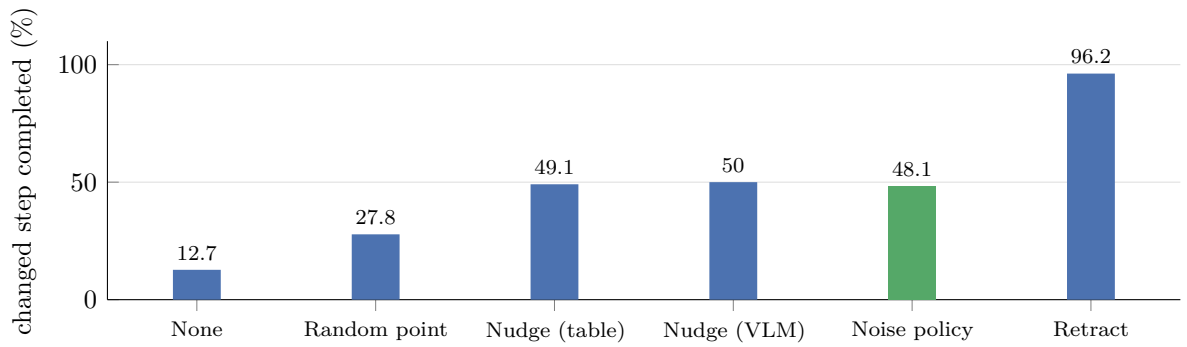



**Figure 3:** Change of mind inside full kitchen missions ("actually, don't do X, do Y"), about 53 missions per bar. *Nudge (table)*: target from a per-task table; *Nudge (VLM)*: the planner points at the target in the image; *Retract*: back to the rest pose, then the new command.

### Results: inside full missions

- Stop, retract and re-command completes 96% of changed steps, the noise policy and the nudge about 50%, no steering 13%.
- Targets pointed by a VLM, often 15 cm off, work as well as exact ones; a random target does not.
- When the change comes in the middle of a grasp, setting the object down before retracting keeps the success rate and places the object instead of dropping it (Table 2). No retraction touched the scene in ≈480 tries.

| Change of mind in the middle of a grasp | Changed step done | Holding an object at the change | Arm contacts per mission |
|---|---|---|---|
| No steering | 8/67 | 2/12 | 3.0 |
| Retract, fingers open | 56/69 | 13/15 | 0.3 |
| Retract, keep the grasp | 48/69 | 8/16 | 0.0 |
| **Set down, then retract** | **54/67** | **11/13** | 1.0 |

**Table 2:** Retraction after a change of mind in the middle of a grasp.

### Results: style preferences

The same noise-and-guidance interface steers style. PrefPI raised the height at which the robot carries objects by 52% at ≈90% success; mixing original demonstrations into each round kept the policy from drifting.

### Insights

- A learned noise policy redirects the expert as well as pointing does, without a target and without touching the expert.
- Changing the expert's weights against a learned critic breaks it; the noise is the safe place to steer.
- Inside missions, a full retraction is the most reliable redirect; setting a held object down first keeps it safe.

### What the literature shows

Flow Reversal Steering [19] steers a frozen flow policy from a rough reference and trains a noise policy on the successful noises. DSRL [20] learns a noise policy with reinforcement learning on real robots, and FlowDAgger [21] with human corrections. SuperMimic [16] trains a controller by back-propagating through one learned world model. Classifier-free guidance [17] works as a policy-improvement operator [18]; PrefPI [22] uses it to steer style from a few labelled rollouts.

## 3 World model

### Roles tested

- **Motion checker**: pick the best of several action chunks for the current command.
- **Motion-level goal reading**: imagine 1.6 s of motion for each command and keep the one closest to a goal photo.
- **Subtask-level goal reading**: imagine the scene at the *end* of each command and keep the one closest to the goal photo.
- **Video prediction** on real household recordings (DROID).

### How the subtask-level model works

1. Encode the current camera view (DINOv3 features).
2. For each candidate command, predict the features of the scene at the end of that command.
3. Compare each prediction with the goal photo.
4. Run the command whose predicted end state is closest.

It is trained on (start view, command → end view) pairs from demonstrations, in 3 minutes on a CPU.

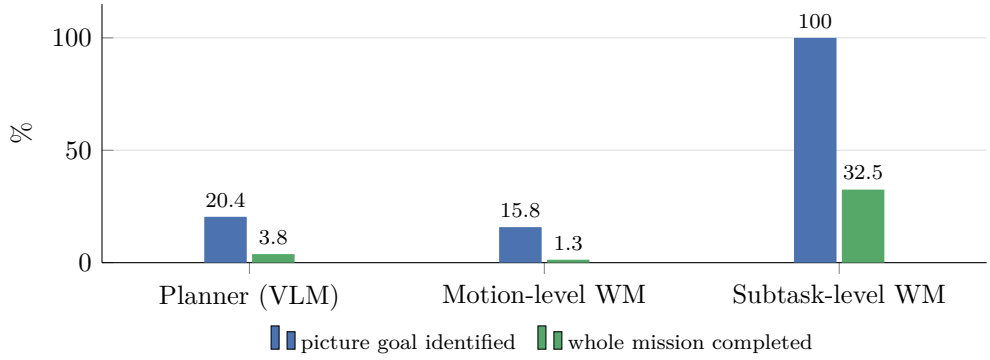



**Figure 4:** Picture-goal step of a three-step kitchen mission ("…then make the kitchen look like this photo"), 80 missions per arm. The robot sees only its camera and the goal photo.

### Results

- The subtask-level model identifies **53/53** picture goals; the planner reading the photo identifies 11/54.
- Whole missions completed: **26/80** with the subtask-level model, 3/80 with the planner.
- Imagining only 1.6 s of motion is not enough: "turn on the stove" and "put the bowl on the stove" look the same for the first second (right command 12% of the time).

- As a motion checker the world model adds nothing: the expert's alternatives for one command are nearly identical.

| Role | Result |
|---|---|
| Motion checker | 57 vs 56/80 steps; with diverse proposals, same as a random pick |
| Motion-level goal reading | right command 12% of the time |
| **Subtask-level goal reading** | **53/53** goals, missions 3/80 → **26/80**; 97% on held-out demonstrations |
| Critic and dynamics for steering | trains the noise policy (§2) |
| Household video prediction (DROID) | FLUX 3 Action best on folding (0.70× the error of holding still); RoboJEPA improves with size (0.81 → 0.66×) and plans 4× faster with gradients |

**Table 3:** World-model results.

### Insights

- The world model is useful at the level of subtasks: imagining what the scene looks like once a command is done.
- At the level of motions it is not: short imagination cannot tell commands apart, and the expert's alternatives barely differ.

### What the literature shows

Latent world models predict future image features instead of pixels: LaWAM [9], RoboJEPA [8] and FLARE [10]. World-action models such as Cosmos 3 [11], FLUX 3 Action [12] and OpenWAM [13] predict video and actions together. DreamerV3 [14], TD-MPC2 [15] and SuperMimic [16] plan or learn inside such models.

## 4 Memory

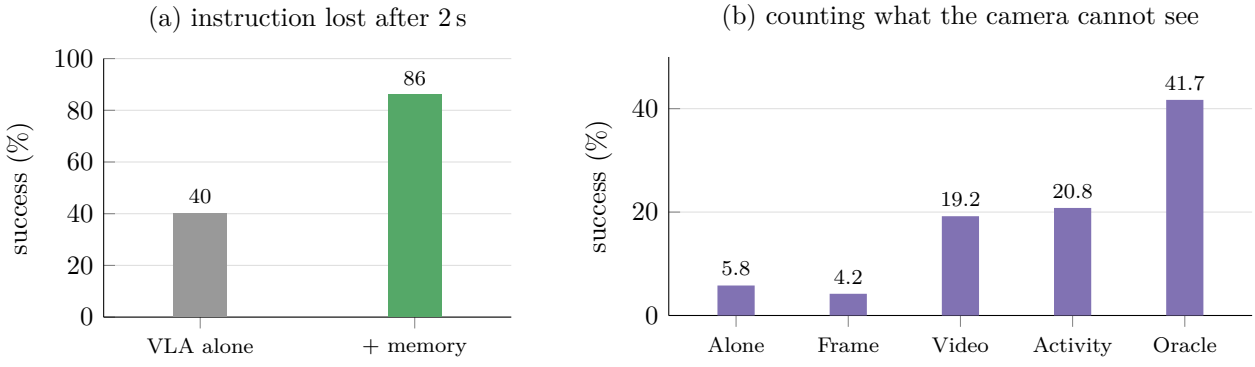



**Figure 5:** (a) An external memory re-issues a lost instruction. (b) "Do it N times" tasks where every repetition looks the same; *Frame*: counting from the current image; *Video*: learned video memory; *Activity*: counting the robot's own grasp–lift–release cycles; *Oracle*: perfect count.

### Results

- **Lost instruction**: an external memory that re-issues the command raises success from 40% to 86%.

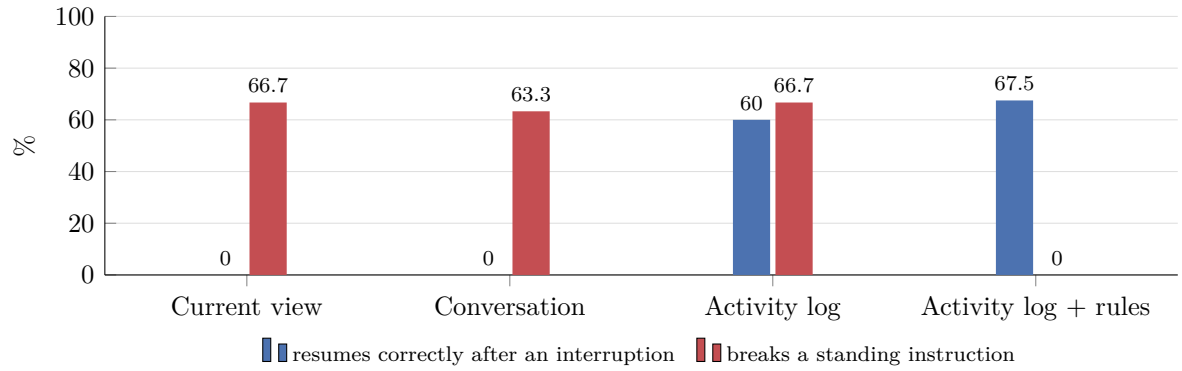



**Figure 6:** What the planner remembers: "First A, then B, then C" → "Wait, X first" → "OK, carry on" (40 episodes), and "Don't open any drawers today" followed by other tasks (30 episodes). Planner: Qwen3-VL-4B.

- **Counting**: when every repetition looks the same, the robot alone succeeds 5.8% of the time; a video memory reaches 19.2% and a counter of its own grasp–lift–release cycles 20.8% (a perfect count gives 41.7%).
- **Interruptions**: only the activity log lets the planner resume at the right step (60%; the conversation alone or the current view: 0%).
- **Standing instructions**: stored as rules that remove forbidden skills, violations drop from 20/30 to 0/30. Kept only in the conversation, they are ignored.
- **Full missions**: without memory, step-1 completion drops from 17/24 to 6/24.

| Memory | Solves | Result | Cost |
|---|---|---|---|
| **Activity log** (plan, skills done, time per skill, interruptions) | resuming after an interruption | 23–27/40 vs 0–7/40, two planners | none |
| **Rules memory** | standing instructions | 20/30 → 0/30 violations | none |
| **Activity counter** (finger and hand sensors) | counts the camera cannot see | 20.8% vs 5.8% | none |
| Video memory (learned) | visual state over time | 19.2%; next step right 74% vs 54% when scenes repeat | small model |
| Progress monitor with stop | ending multi-step tasks cleanly | 2/40 → 18–19/40 | small model |

**Table 4:** Memory types tested.

### Insights

- The motion expert only sees the present; memory sits outside it and feeds the planner, so it works with any motion expert.
- Cheap structured memories (activity log, rules, counters) cover most needs; a learned video memory helps where the scene hides progress.

### What the literature shows

LIBERO-Mem [23] and MIKASA-Robo-VLA [24] show that VLAs acting on the present frame fail tasks that need history. 3D-Mem [25] keeps snapshots of key events for embodied reasoning.

# 5 Planner

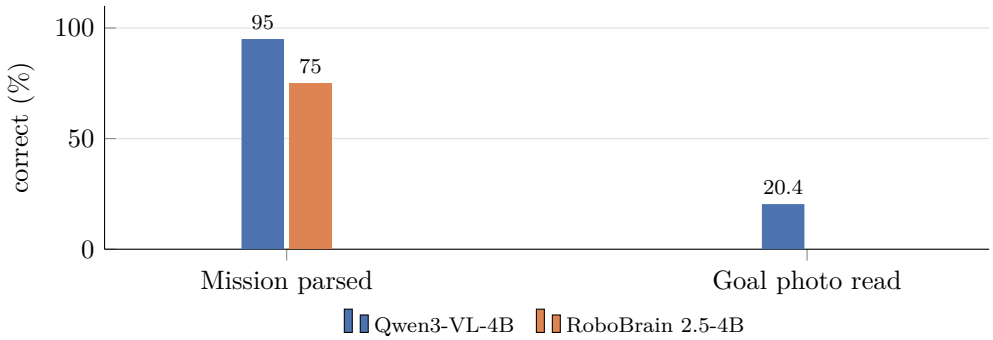



**Figure 7:** Two 4B planners on the kitchen missions: turning the spoken mission into skills (80 missions) and reading a goal photo (Qwen3-VL-4B, 54 missions).

### Results

- Qwen3-VL-4B parses 76/80 spoken missions, RoboBrain 2.5-4B 60/80.
- Goal photos are read poorly (11/54); the subtask-level world model reads them (§3).
- Standing instructions kept in the conversation are ignored; rules memory handles them (§4).
- Free-text sub-goals outside the motion expert's training vocabulary do not help (2/32 vs 4/32 on RoboCasa365).

### Insights

- Qwen3-VL-4B is the better base planner.
- The planner works best when paired with the world model for pictures, rules memory for constraints, and sub-goals the motion expert knows.

### What the literature shows

Embodied VLMs such as RoboBrain 2.0 [26] and general VLMs such as Qwen3-VL [27] decompose instructions and point at targets. $\pi_{0.5}$ [1] and $\pi_{0.7}$ [2] drive their action expert with subtasks from a high level.

# 6 Lessons for training

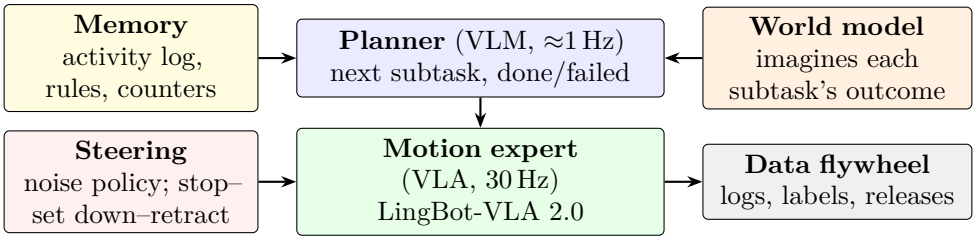
- **Own rollouts alone cause drift**: one fine-tuning round on its own rollouts made G0.5 11% slower; mixing about 50% original demonstrations into each round removed the drift.
- **The noise is the safe place to improve**: the same learned critic that trained a useful noise policy broke the expert when applied to its weights.
- **Modules outside the expert are independent of it**: memory, planner, world model and steering keep working when the expert changes.

### What the literature shows

Pretrained VLAs keep their skills over repeated fine-tuning [7]. A recent two-arm household study [29] scaled from 1,500 hours of demonstrations to corrections collected while the robot works.

# Part II Proposal

## 7 Architecture




The planner turns requests into short subtasks, using memory and the world model; the motion expert executes each subtask; the steering module redirects it when the user changes their mind; every run feeds the data flywheel.

## 8 Motion expert: LingBot-VLA 2.0

**What LingBot brings**

- **Apache-2.0 licence**: free to use, modify and ship in a commercial product.
- **Two arms, many robots**: one 55-dimensional action space with slots for arms, grippers, base and waist. OpenArm 2 maps onto slots the model already learned in pretraining.
- **Richer perception**: short video input and a depth teacher during training, which helps grasping and contact.
- **Runs on the robot**: ≈130 ms per action chunk on one RTX 4090D.
- **Same family as G0.5** (flow matching, 10-step sampler): the steering methods of Part I apply directly.

| | **G0.5** | **LingBot-VLA 2.0** | $\pi_{0.5}$ / GR00T N1.7 |
|---|---|---|---|
| Size, backbone | Qwen3.5 VLM + 635M action expert | 6B, Qwen3-VL-4B | 3B each |
| Robot bodies | single arm (LIBERO checkpoint) | two arms, base, waist; many robots | multi-robot; GR00T has an OpenArm 2 checkpoint |
| Perception | camera frames | camera frames + video, depth-trained | camera frames |

**Table 5:** Motion-expert candidates.

**Plan**

- **Simulation comparison**: LingBot, $\pi_{0.5}$ and GR00T trained on RoboCasa365 with the same budget, evaluated with the same harness.
- **Bake-off on the OpenArm 2 data**: the three policies trained on our recordings, 10 tasks × 20 trials each on the robot.

### What it means for the real robot

A single consumer GPU on the robot runs LingBot in real time. Two-arm control and depth-aware perception match OpenArm 2 and household objects, and the licence lets the product ship.

### What the literature shows

LingBot-VLA 2.0 [6] is a 6B model with a cross-embodiment action space, trained for two-arm robots. GR00T N1.7 [5] has an OpenArm 2 checkpoint.

## 9 Steering module

- **Noise policy as the learned steering module**, trained per skill from the robot's own rollouts.
- **Stop, set down, retract, re-command** as the fallback.
- **Retraction as teacher**: a student network learns from both cameras and the robot state how far the robot needs to back off in each situation, from not at all to a full retraction, to keep the fallback's reliability with less detour.
- **Style preferences** through the same interface (PrefPI).

### What it means for the real robot

Users correct robots all the time. The noise policy needs no pointing and adds little latency. It is refreshed per skill from the robot's logs while the motion expert stays the same, and the retraction catches the cases it misses.

## 10 World model in the pipeline

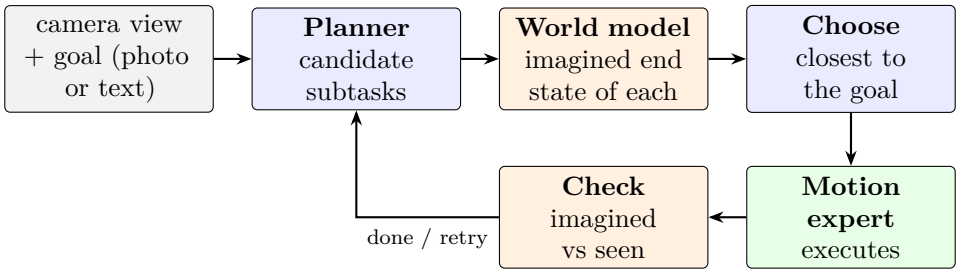



**Figure 8:** The world model beside the planner.

### What it adds

1. **Picture goals**: "make it look like this" becomes a sequence of subtasks.
2. **Plan checking**: imagine a plan's outcome before running it.
3. **Success detection**: compare the imagined end state with the camera view; a mismatch triggers a retry or a question to the user.
4. **Training signal**: the critic and dynamics model behind the noise policies.
5. **Data curation**: score and filter logged episodes.

### What it means for the real robot

A user can show a photo ("set the table like this"). The robot predicts the outcome of each subtask and checks it after doing it, which is the basis for success detection and later for safety checks. The model is small, ours, and trained on the robot's logs.

# 11 Memory

- **Activity log**: plan, skills done, time per skill, interruptions; read by the planner to resume.
- **Rules memory**: standing instructions stored as forbidden skills.
- **Activity counter**: repetitions counted from the robot's own finger and hand sensors.
- **Video memory**: for tasks where the scene hides progress.
- **Progress monitor**: issues the next step and stops the robot when the task is done.

**What it means for the real robot**

Household work is interrupted, comes with standing preferences ("never put knives in that drawer") and involves counts (three scoops, five shirts). The activity log, rules and counters cost nothing to run.

# 12 Planner

Qwen3-VL-4B is the baseline. Options to evaluate on the same missions:

| Option | What it brings | Cost |
|---|---|---|
| Qwen3-VL-4B fine-tuned (LoRA) on OpenArm subtask annotations and corrected plans | knowledge of our tasks and robot | ≈50–100 H200-h |
| Larger open VLMs (8B–32B) | better reasoning on long or vague requests | latency, GPU on the robot |
| Cloud models | strongest reasoning | latency, cost, privacy |
| Skill-vocabulary outputs (choose among the expert's trained subtasks) | no sub-goals the expert cannot execute | list of skills |
| Planner + subtask-level world model | picture goals, plan checks, success detection | small model |

**What it means for the real robot**

The planner turns everyday speech into subtasks and handles corrections. It was the most frequent failure point in our missions, so improving it raises the reliability of the whole system.

# 13 Data and end-to-end training

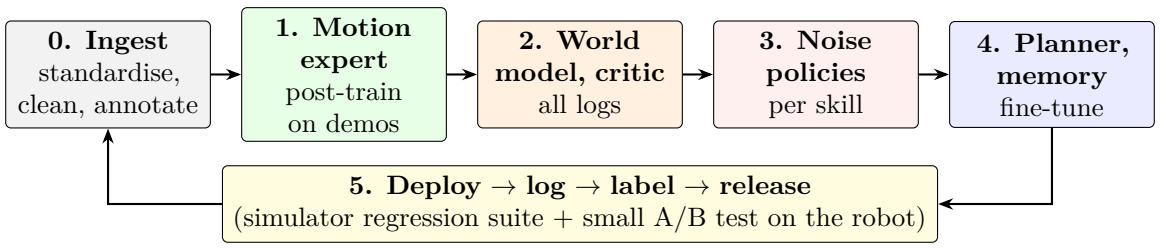



**Figure 9:** The training flywheel.

| Module | Data | Labels | Scale in our runs |
|---|---|---|---|
| Motion expert | teleoperated OpenArm 2 demonstrations cut into 2–10 s subtasks; varied start states; **correction episodes** (operator takes over, then hands back) | subtask text (VLM proposal, checked by a person); success | 50 demonstrations per task (LIBERO) |
| Noise policy | robot rollouts with mid-task changes of instruction; retraction rollouts | success of the new subtask | 600–1,400 episodes |
| World model and critic | all logs, failures included: (view, subtask → end view) pairs; video and actions | automatic from logs | 382 demonstrations for 10 commands |
| Memory | event timelines; interruption and rule scenarios; repetition tasks | automatic from logs; scripted scenarios | 100 demonstrations per task |
| Planner | subtask annotations, corrected plans, goal photos | VLM proposals checked by a person | – |
| Style preferences | rollouts ranked by an operator | rankings | ≈150 labels per preference (PrefPI) |

**Table 7:** Data per module. One recording session feeds several modules.

### Training stages

0. **Ingest**: one control rate, camera names, joint order and gripper convention; broken episodes removed; episodes split into subtasks with language.
1. **Motion expert**: post-train LingBot on the OpenArm demonstrations, correction episodes included, with original demonstrations mixed into every later round.
2. **World model and critic**: trained on all logs, failures included.
3. **Noise policies**: per skill, imitate successful noises, then improve through the world model and critic.
4. **Planner and memory**: fine-tune the planner on annotated subtasks and corrected plans; build memories from event timelines.
5. **Deploy, log, label, release**: take-overs and success labels flow back into steps 0–4; a new version ships after the simulator regression suite and a small A/B test on the robot.

| Stage | Compute |
|---|---|
| Motion-expert comparison (RoboCasa365) and bake-off on OpenArm data | 600–1,000 H200-h |
| World model and critic on OpenArm data | 100–200 H200-h |
| Planner fine-tune (Qwen3-VL-4B, LoRA) | 50–100 H200-h |
| Noise policies, memory modules, evaluation | ≈400 T4-h |

**Table 8:** Compute per stage.

### What it means for the real robot

Every hour of operation produces training data for all modules: operator take-overs become correction data, failures train the world model and critic, and each release is tested before it ships.

## Requests

1. **Compute**: ≈600–1,000 H200-hours for the motion-expert comparison, ≈100–200 for the world model and critic, ≈50–100 for the planner fine-tune.
2. **Data**: access to the OpenArm 2 recordings.

## References


[1] K. Black et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. CoRL (PMLR 305), 2025.

[2] Physical Intelligence. $\pi_{0.7}$. arXiv:2604.15483, 2026.

[3] Galaxea. G0.5. arXiv:2608.11739, 2026. https://github.com/OpenGalaxea/GalaxeaVLA

[4] Awomo-0.5. https://github.com/Awomo-WestlakeDI/Awomo-0.5

[5] NVIDIA. Isaac GR00T N1.7. https://github.com/NVIDIA/Isaac-GR00T

[6] Robbyant. LingBot-VLA 2.0. 2026. https://github.com/robbyant/lingbot-vla-v2

[7] Pretrained vision-language-action models are surprisingly resistant to forgetting in continual learning. arXiv:2603.03818, 2026.

[8] RoboJEPA: Scaling robotic latent world models. arXiv:2610.10515, 2026.

[9] LaWAM. arXiv:2606.15768, 2026.

[10] FLARE: Robot learning with implicit world modeling. arXiv:2505.15659, 2025.

[11] NVIDIA. Cosmos 3. https://huggingface.co/collections/nvidia/cosmos3

[12] Black Forest Labs. FLUX 3 Action. https://huggingface.co/black-forest-labs/flux-3-action-droid

[13] OpenWAM: An open, modular exploration towards systematic world-action model pretraining. arXiv:2609.07398, 2026.

[14] D. Hafner et al. Mastering diverse domains through world models (DreamerV3). 2023.

[15] N. Hansen et al. TD-MPC2: Scalable, robust world models for continuous control. ICLR, 2024.

[16] Q. Liao et al. SuperMimic: Scaling humanoid whole-body control with one differentiable world model. CoRL, 2026. arXiv:2610.28480.

[17] J. Ho, T. Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022.

[18] K. Frans, S. Park, P. Abbeel, S. Levine. Diffusion guidance is a controllable policy improvement operator. arXiv:2505.23458, 2025.

[19] Improving robotic generalist policies via flow reversal steering. arXiv:2606.13675, 2026.

[20] A. Wagenmaker et al. Steering your diffusion policy with latent space reinforcement learning. CoRL, 2025.

[21] FlowDAgger: Human-in-the-loop adaptation of generative robot policies in latent space. arXiv:2607.08877, 2026.

[22] S. Rho, W. Kim, D. Xu, S. Ha. PrefPI: Preference-guided steering into out-of-distribution behaviors. arXiv:2609.40165, 2026.

[23] Chung et al. LIBERO-Mem. arXiv:2511.11478, 2025.

[24] MIKASA-Robo-VLA: Benchmarking memory in VLA models for long-horizon manipulation. arXiv:2610.00604, 2026.

[25] 3D-Mem: 3D scene memory for embodied exploration and reasoning. CVPR, 2025. arXiv:2411.17735.

[26] BAAI. RoboBrain 2.0 technical report. arXiv:2507.02029, 2025.

[27] Qwen Team. Qwen3-VL. 2025.

[28] RoboCasa365: A large-scale simulation framework for training and benchmarking generalist robots. arXiv:2603.04356, 2026.

[29] Scaling bimanual household manipulation from 1,500 hours of demonstrations to on-policy corrections. arXiv:2609.03591, 2026.